

Particle System 구현 설명서

1. 목표

이번 작업의 목표는 Cascade 스타일의 CPU 기반 Particle System을 엔진에 통합하는 것이다. 에디터에서는 파티클 시스템 에셋을 만들고 Sprite, Mesh, Beam, Ribbon 타입의 Emitter를 편집할 수 있어야 하며, 런타임에서는 `UParticleSystemComponent`가 월드에 배치되어 시뮬레이션과 렌더링을 담당한다.

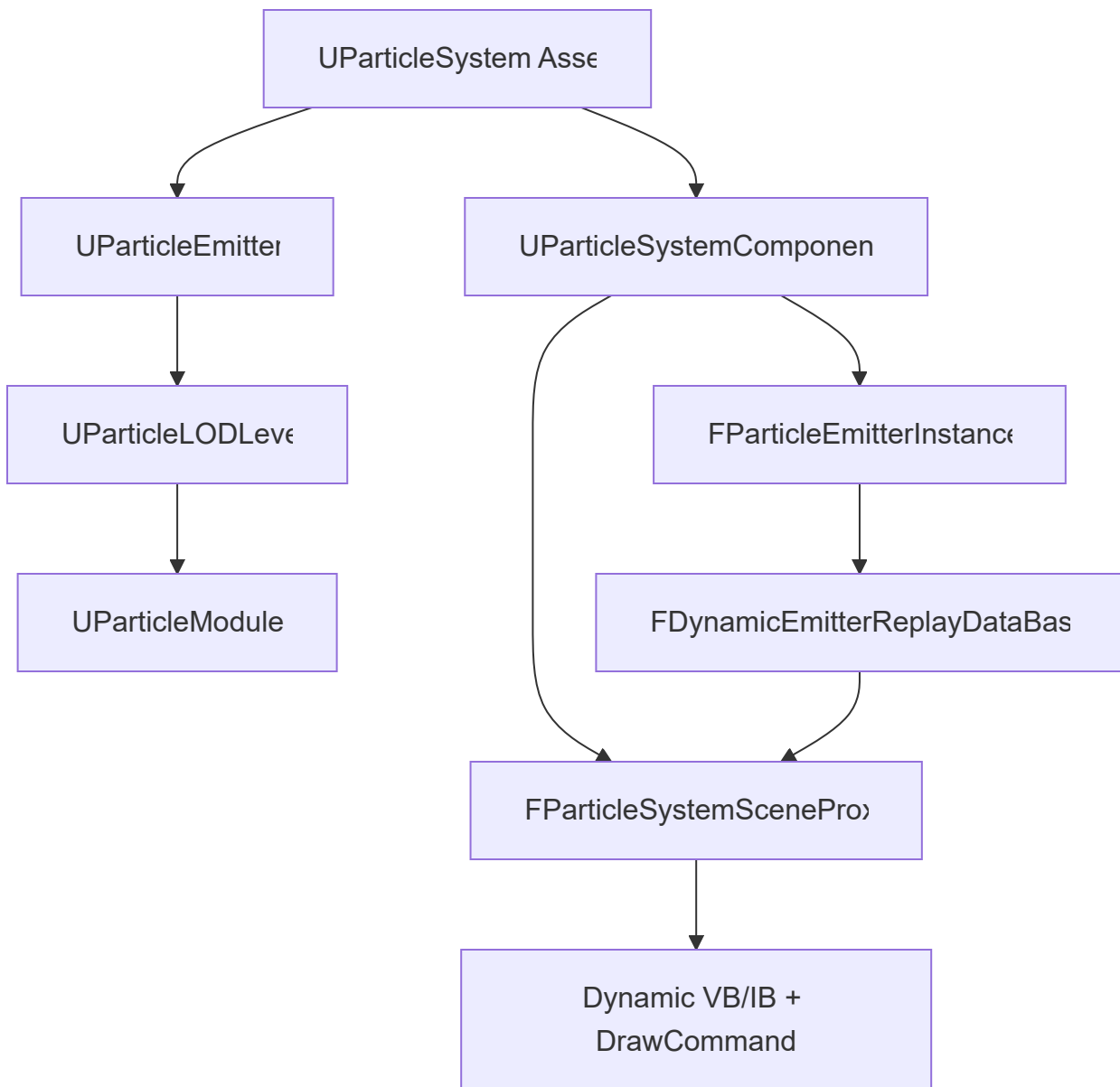
핵심 요구사항은 다음과 같다.

- Particle System Editor 구현
- Sprite Emitter, Mesh Emitter 지원
- Particle System Component 배치 및 렌더링
- Show Flag, Stat 연동
- `UParticleSystem`, `UParticleEmitter`, `UParticleLODLevel`, `UParticleModule` 기반의 코어 구조
- Symbol Server 구축 스크립트 제공
- 선택 기능: LOD, Collision/Event, Beam, Ribbon, Curve Editor

이 시스템은 단순히 화면에 파티클을 띄우는 코드가 아니라, 에셋 데이터, 인스턴스 시뮬레이션, 렌더 스냅샷, 렌더 프록시, 에디터 편집 UI가 분리된 구조로 설계되어 있다. 이 분리는 유지보수성과 확장성을 위해 중요하다.

2. 전체 구조

Particle System은 크게 네 계층으로 나뉜다.



각 계층의 책임은 다음과 같다.

| 계층 | 대표 타입 | 책임 |

|---|---|---|

| 에셋 | UParticleSystem | 여러 Emitter와 LOD 거리 정보를 가진 공유 에셋 |

| Emitter 템플릿 | UParticleEmitter | LOD 목록, 모듈 오프셋, 파티클 stride 계산 |

| LOD | UParticleLODLevel | 특정 LOD에서 사용할 Required/Spawn/Update/TypeData/Event 모듈 집합 |

| Module | `UParticleModule` 파생 클래스 | Spawn/Update/FinalUpdate 단계별 파티클 속성 변경 |

| Component | `UParticleSystemComponent` | 월드에 배치되는 실행 인스턴스, Tick, LOD 선택, 이벤트 디스패치 |

| Emitter Instance | `FParticleEmitterInstance` | 실제 파티클 메모리, 생성/소멸/업데이트, `ReplayData` 생성 |

| Render Snapshot | `FDynamicEmitterReplayDataBase` 파생 | 렌더 스레드에 넘길 프레임 단위 복사 데이터 |

| Scene Proxy | `FParticleSystemSceneProxy` | 렌더용 정점/인덱스 패킹, draw section 구성 |

에셋과 런타임 인스턴스를 분리한 이유는 여러 Actor가 같은 `UParticleSystem` 에셋을 공유하더라도 각 Component의 재생 시간, 활성 파티클, Collision 이벤트, LOD 상태는 서로 달라야 하기 때문이다.

3. Editor & Rendering

3.1 Particle System Editor

Particle System Editor는 `FParticleEditorWidget` 이 담당한다.

주요 파일:

- `KraftonEngine/Source/Editor/UI/Asset/ParticleEditorWidget.h`
- `KraftonEngine/Source/Editor/UI/Asset/ParticleEditorWidget.cpp`

주요 기능:

- `UParticleSystem` 에셋 열기
- Emitter 추가/삭제/이름 변경/순서 이동
- Emitter TypeData 변경

- Sprite

- Mesh

- Beam

- Ribbon

- LOD 추가/삭제 및 LOD 거리 편집
- Required Module 편집

- Material

- Screen Alignment

- Sort Mode

- SubUV Columns/Rows

- Alpha Source/Threshold/Power

- Color Intensity

- Spawn, Lifetime, Location, Velocity, Size, Color 계열 모듈 편집
- Collision, Event Generator, Event Receiver 편집
- Preview World에서 `UParticleSystemComponent` 를 생성해 실시간 미리보기

에디터는 파티클 에셋 자체를 직접 수정하고, 수정 후 `ApplyEmitterEdit()` 또는 Preview 재시작 흐름을 통해 시뮬레이션 인스턴스와 렌더 데이터를 갱신한다.

3.2 Curve Editor

Curve Editor는 선택 요구사항이지만 현재 Editor 구조에 포함되어 있다.

`FParticleDistributionFloat`, `FParticleDistributionVector` 는 다음 모드를 지원한다.

- Constant
- Uniform
- Constant Curve
- Uniform Curve

Curve Editor는 선택된 Module에서 편집 가능한 Distribution Track을 수집하고, 키 추가/삭제/드래그, 시간/값 직접 입력을 제공한다. 예를 들어 Spawn Rate, Lifetime, Start Size, Start Velocity, Color Over Life 같은 값은 Curve Track으로 표시될 수 있다.

관련 함수:

- `RenderCurvePanel()`
- `CollectModuleCurveTracks()`
- `RenderFloatDistributionControls()`
- `RenderVectorDistributionControls()`

3.3 Content Browser / 월드 배치

Particle System 에셋은 Content Browser에서 생성하고 열 수 있다.

관련 파일:

- `KraftonEngine/Source/Editor/Subsystem/AssetFactory.cpp`
- `KraftonEngine/Source/Editor/UI/ContentBrowser/ContentBrowser.cpp`
- `KraftonEngine/Source/Editor/UI/ContentBrowser/ContentBrowserElement.cpp`

에셋을 뷰포트로 Drag & Drop하면 `AEmitter` 또는 Particle Component를 가진 Actor가 생성되고, `UParticleSystemComponent::Template`에 해당 에셋이 연결된다.

관련 흐름:



3.4 Sprite Emitter Rendering

Sprite Emitter는 각 파티클을 카메라를 향하는 quad로 확장해 렌더링한다.

렌더 데이터 타입:

- `FDynamicSpriteEmitterReplayData`
- `FDynamicSpriteEmitterData`
- `FParticleSpriteVertex`

렌더 프록시 흐름:

1. `FParticleEmitterInstance::GetReplayData()` 가 활성 파티클을 `FParticleDataContainer`에 복사한다.
2. `UParticleSystemComponent::TickComponent()` 가 `ReplayData`를 `FDynamicSpriteEmitterData`로 감싼다.
3. `FParticleSystemSceneProxy::UpdateDynamicData()`가 동적 데이터를 소유한다.
4. `FParticleSystemSceneProxy::UpdatePerViewport()`에서 카메라 기준 정렬과 패킹을 수행한다.
5. `FSpriteParticlePacker`가 파티클당 4개 vertex와 6개 index를 동적 VB/IB에 업로드한다.
6. AlphaBlend Pass에서 Material, BlendState, Particle Constant Buffer를 바인딩해 그린다.

Sprite는 SubUV, Screen Alignment, Alpha 설정을 Required Module에서 받아 렌더 상수 버퍼로 전달한다.

3.5 Mesh Emitter Rendering

Mesh Emitter는 `UParticleModuleTypeDataMesh`를 통해 Static Mesh를 지정한다.

핵심 타입:

- `UParticleModuleTypeDataMesh`
- `FDynamicMeshEmitterReplayData`
- `FDynamicMeshEmitterData`
- `FMeshParticleInstanceVertex`

Mesh Emitter는 파티클마다 mesh geometry를 복제하지 않고, per-particle transform/color를 instance buffer에 패킹한다. 렌더러는 Static Mesh의 기존 geometry buffer를 재사용하고, 파티클 정보만 별도 instance vertex buffer로 업로드한다.

이 방식은 CPU 메모리와 업로드 비용을 줄이고, 파티클 수가 많아져도 draw command 수를 안정적으로 유지하기 위한 구조다.

3.6 Beam / Ribbon Rendering

선택 학습 항목인 Beam과 Ribbon도 별도의 TypeData와 Emitter Instance로 분리되어 있다.

Beam 관련 파일:

- `KraftonEngine/Source/Engine/Particle/ParticleBeamInstances.h`

- `KraftonEngine/Source/Engine/Particle/ParticleBeamInstances.cpp`
- `KraftonEngine/Source/Engine/Particle/TypeData/ParticleModuleTypeDataBeam2.h`
- `KraftonEngine/Source/Engine/Particle/BeamModule/ParticleModuleBeamSource.h`
- `KraftonEngine/Source/Engine/Particle/BeamModule/ParticleModuleBeamTarget.h`
- `KraftonEngine/Source/Engine/Particle/BeamModule/ParticleModuleBeamNoise.h`

Ribbon 관련 파일:

- `KraftonEngine/Source/Engine/Particle/RibbonEmitterInstance.h`
- `KraftonEngine/Source/Engine/Particle/RibbonEmitterInstance.cpp`
- `KraftonEngine/Source/Engine/Particle/TypeData/ParticleModuleTypeDataRibbon.h`

렌더 프록시에는 `FBeamParticlePacker`, `FRibbonParticlePacker` 가 있으며, 각각 CPU에서 카메라 기준 geometry를 확장해 동적 VB/IB에 업로드한다.

4. Engine Core

4.1 UParticleSystem

`UParticleSystem` 은 여러 `UParticleEmitter` 를 보유하는 에셋이다.

파일:

- `KraftonEngine/Source/Engine/Particle/ParticleSystem.h`
- `KraftonEngine/Source/Engine/Particle/ParticleSystem.cpp`

주요 데이터:

```
class UParticleSystem : public UFXSystemAsset
{
public:
    TArray<UParticleEmitter*> Emitters;
```

```

int32 GetLODCount() const;

int32 CreateLOD(float Distance = -1.0f);

bool RemoveLOD(int32 LODIndex);

void NormalizeLODData();

private:

    TArray<float> LODDistances = { 0.0f };

};

```

`NormalizeLODData()` 는 시스템이 가진 LOD 개수와 각 Emitter의 LOD 개수를 맞춘다. LOD 개수가 틀어지면 런타임에서 특정 Emitter만 잘못된 LOD를 참조할 수 있으므로, 에셋 로드/초기화 시점에 정규화하는 구조가 필요하다.

4.2 UParticleEmitter

`UParticleEmitter` 는 Emitter 템플릿이다.

파일:

- `KraftonEngine/Source/Engine/Particle/ParticleEmitter.h`
- `KraftonEngine/Source/Engine/Particle/ParticleEmitter.cpp`

주요 책임:

- `LODLevels` 보관
- 모듈 역할 분류
- 최대 활성 파티클 수 계산
- 모듈별 payload offset 계산
- per-particle stride 계산
- per-instance payload 크기 계산

중요 함수:

- `ClassifyModulesByRole()`
- `CalculateMaxActiveParticleCount()`
- `CacheEmitterModuleInfo()`
- `CalculateTypeParticleSizeAndOffsets()`
- `CalculatePerParticleSizeAndOffsets()`
- `CalculatePerInstanceParticleSizeAndOffset()`

`CacheEmitterModuleInfo()` 는 각 Module이 요구하는 per-particle payload와 per-instance payload를 계산한다. 이 계산이 틀리면 Spawn/Update 루프에서 payload offset이 어긋나므로 메모리 오염이나 잘못된 렌더 결과가 발생한다.

4.3 UParticleLODLevel

`UParticleLODLevel` 은 한 LOD 단계에서 사용할 모듈 집합이다.

파일:

- `KraftonEngine/Source/Engine/Particle/ParticleLODLevel.h`
- `KraftonEngine/Source/Engine/Particle/ParticleLODLevel.cpp`

주요 데이터:

```
class UParticleLODLevel : public UObject
{
public:
    int32 Level = 0;

    uint32 bEnabled : 1 = false;

    UParticleModuleRequired* RequiredModule = nullptr;

    TArray<UParticleModule*> Modules;

    UParticleModuleTypeDataBase* TypeDataModule = nullptr;
```

```

UParticleModuleSpawn* SpawnModule = nullptr;

UParticleModuleEventGenerator* EventGenerator = nullptr;

TArray<UParticleModuleSpawnBase*> SpawningModules;

TArray<UParticleModule*> OnSpawnModules;

TArray<UParticleModule*> UpdateModules;

TArray<UParticleModule*> FinalUpdateModules;

TArray<UParticleModuleEventReceiverBase*> EventReceiverModules;

};

```

일반 `Modules` 배열만 순회하지 않고 역할별 배열을 따로 캐싱하는 이유는 Tick 중 매 프레임 RTTI나 타입 분기를 반복하지 않기 위해서다. 파티클 시스템은 프레임 타이밍에 민감하므로, 편집/빌드 단계에서 분류하고 런타임은 분류 결과를 빠르게 순회하는 편이 안정적이다.

4.4 UParticleModule

`UParticleModule` 은 파티클 동작의 확장 단위다.

파일:

- `KraftonEngine/Source/Engine/Particle/ParticleModule.h`
- `KraftonEngine/Source/Engine/Particle/ParticleModules.cpp`

기본 인터페이스:

```

class UParticleModule : public UObject
{
public:

    virtual void Spawn(const FSpawnContext& Context);

```

```

    virtual void Update(const FUpdateContext& Context);

    virtual void FinalUpdate(const FUpdateContext& Context);

    virtual uint32 RequiredBytes(UParticleModuleTypeDataBase* TypeData =
    nullptr);

    virtual uint32 RequiredBytesPerInstance();

    virtual EModuleType GetModuleType() const;

};

```

주요 Module:

| Module | 역할 |

|---|---|

| UParticleModuleRequired | Material, ScreenAlignment, SortMode, SubUV, Alpha 등 Emitter
필수 설정 |

| UParticleModuleSpawn | 초당 생성량과 Burst 생성량 계산 |

| UParticleModuleLifetime | 파티클 수명 설정 |

| UParticleModuleLocation | 초기 위치 설정 |

| UParticleModuleVelocity | 초기 속도 설정 |

| UParticleModuleSize | 초기 크기 설정 |

| UParticleModuleColor | 초기 색상/알파 설정 |

| UParticleModuleColorOverLife | 수명 진행도 기준 색상 변화 |

| UParticleModuleColorScaleOverLife | 기존 색상에 대한 scale 적용 |

| `UParticleModuleAcceleration` | 속도에 가속도 적용 |

| `UParticleModuleOrbit` | orbit offset/rotation payload 적용 |

| `UParticleModuleCollision` | 월드 trace 기반 충돌 처리 |

| `UParticleModuleEventGenerator` | Spawn/Death/Collision/Burst 이벤트 생성 |

| `UParticleModuleEventReceiverSpawn` | 이벤트를 받아 추가 파티클 생성 |

| `UParticleModuleTypeDataMesh` | Mesh Emitter 타입 지정 |

Module은 Spawn, Update, FinalUpdate 중 필요한 단계만 구현한다. 이 구조는 새 기능을 추가할 때 기존 Emitter Instance의 메인 루프를 크게 바꾸지 않고 Module 단위로 확장할 수 있게 한다.

5. Particle System Component

5.1 Component 책임

`UParticleSystemComponent` 는 월드에 배치되는 실행 단위다.

파일:

- `KraftonEngine/Source/Engine/Component/ParticleSystemComponent.h`
- `KraftonEngine/Source/Engine/Component/ParticleSystemComponent.cpp`

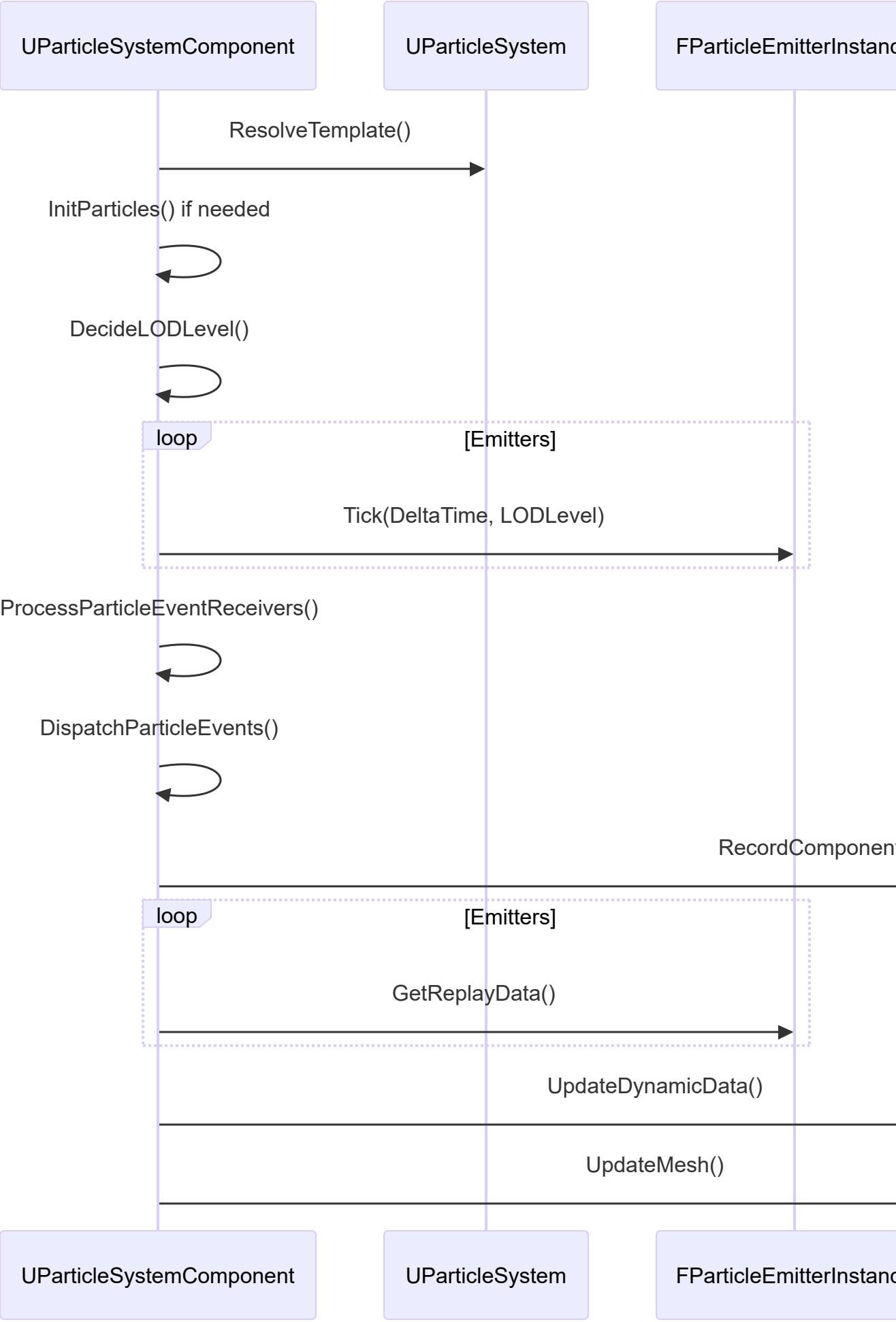
주요 책임:

- `Template` Particle System 에셋 참조
- Emitter Instance 생성/소멸
- Tick에서 LOD 결정
- Emitter Instance Tick 호출
- Collision/Event 큐 수집 및 디스패치
- `ReplayData`를 렌더 프록시로 전달
- Show Particles, Sort Priority 반영

중요 데이터:

```
TSoftObjectPtr<UParticleSystem> Template;  
  
TArray<FParticleEmitterInstance*> EmitterInstances;  
  
int32 LODLevel = 0;  
  
int32 ForcedLODLevel = -1;  
  
TArray<float> LODDistances;  
  
TArray<FParticleEventCollideData> CollisionEvents;
```

5.2 Tick 흐름



UParticleSystemComponent 는 렌더링을 직접 수행하지 않는다. Component는 시뮬레이션 결과를 ReplayData로 만들고, 렌더링은 Scene Proxy가 담당한다. 이 분리는 시뮬레이션과 렌더 파이프라인의 결합도를 낮춘다.

5.3 Emitter Instance

FParticleEmitterInstance 는 실제 파티클 메모리를 소유한다.

파일:

- KraftonEngine/Source/Engine/Particle/ParticleEmitterInstances.h
- KraftonEngine/Source/Engine/Particle/ParticleEmitterInstances.cpp

주요 데이터:

```
uint8* ParticleData = nullptr;

uint16* ParticleIndices = nullptr;

uint8* InstanceData = nullptr;

int32 ParticleSize = 0;

int32 ParticleStride = 0;

int32 ActiveParticles = 0;

int32 MaxActiveParticles = 0;

float SpawnFraction = 0.0f;

FParticleDataContainer DataContainer;
```

ParticleData 는 FBaseParticle 뒤에 Module payload가 이어지는 구조다. ParticleIndices 는 활성 파티클 순서와 Kill/Sort를 위한 indirection array로 사용된다.

Spawn 흐름은 다음과 같다.

1. Spawn Module이 이번 프레임 생성 수를 계산한다.
2. SpawnParticles() 가 빈 slot을 잡는다.
3. PreSpawn() 이 기본 위치/속도/시간 값을 초기화한다.
4. LOD의 OnSpawnModules 를 순회하며 초기 속성, payload를 설정한다.
5. TypeData Spawn이 필요한 경우 Mesh/Beam/Ribbon 전용 데이터를 초기화한다.
6. PostSpawn() 이 최종 보정과 이벤트 생성을 처리한다.

Update 흐름은 다음과 같다.

1. 일반 Update Module 실행
2. 파티클 위치/속도/수명 업데이트
3. PostUpdate Module 실행
4. FinalUpdate Module 실행
5. Lifetime이 끝난 파티클 Kill
6. Collision/Event 처리

6. 렌더 데이터 구조

6.1 FParticleDataContainer

FParticleDataContainer 는 렌더러로 전달할 파티클 byte block을 소유한다.

파일:

- KraftonEngine/Source/Engine/Render/Particle/ParticleDynamicData.h
- KraftonEngine/Source/Engine/Render/Particle/ParticleDynamicData.cpp

```
struct FParticleDataContainer
{
    int32 MemBlockSize = 0;

    int32 ParticleDataNumBytes = 0;

    int32 ParticleIndicesNumShorts = 0;

    uint8* ParticleData = nullptr;
```



```
uint16* ParticleIndices = nullptr;

};
```

`ParticleData` 와 `ParticleIndices` 를 하나의 memory block에 배치하는 이유는 allocation 수를 줄이고, 프레임 단위 스냅샷 복사를 단순화하기 위해서다.

6.2 FDynamicEmitterReplayDataBase

`ReplayData`는 렌더러가 프레임 중 참조할 immutable snapshot이다.

주요 파생 타입:

- `FDynamicSpriteEmitterReplayData`
- `FDynamicMeshEmitterReplayData`
- `FDynamicBeamEmitterReplayData`
- `FDynamicRibbonEmitterReplayData`

공통 필드:

```
EDynamicEmitterType eEmitterType;

int32 ActiveParticleCount;

int32 ParticleStride;

FParticleDataContainer DataContainer;

FVector Scale;

EParticleSortMode SortMode;

uint16 EmitterSortPriority;
```

Emitter Type에 따라 Material, Mesh, Beam list, Ribbon trail point 등 추가 데이터가 붙는다.

6.3 Scene Proxy

`FParticleSystemSceneProxy` 는 `UParticleSystemComponent` 의 렌더 표현이다.

파일:

- `KraftonEngine/Source/Engine/Render/Proxy/ParticleSystemSceneProxy.h`
- `KraftonEngine/Source/Engine/Render/Proxy/ParticleSystemSceneProxy.cpp`

주요 내부 packer:

| `Packer` | 역할 |

|---|---|

| `FSpriteParticlePacker` | Sprite 파티클을 quad vertex/index로 확장 |

| `FMeshParticlePacker` | Mesh 파티클 instance buffer 생성 |

| `FBeamParticlePacker` | Beam을 camera-facing strip geometry로 확장 |

| `FRibbonParticlePacker` | Ribbon trail을 strip geometry로 확장 |

Proxy는 `DynamicData` 를 소유하고, 매 프레임 `PackParticles()` 에서 Emitter Type에 맞는 packer 를 호출한다.

정렬은 두 단계로 나뉜다.

- Emitter 내부 정렬: `SortMode` 에 따라 파티클 index를 카메라 기준 정렬
- Emitter 간 정렬: `EmitterSortPriority` 로 섹션 draw 순서 조정

7. Show Flag

Viewport Show Flag에는 `bParticles` 가 추가되어 있다.

파일:

- `KraftonEngine/Source/Engine/Render/Types/ViewTypes.h`

- `KraftonEngine/Source/Editor/UI/Toolbar/ViewportToolbar.cpp`
- `KraftonEngine/Source/Engine/Render/Pipeline/RenderCollector.cpp`

FShowFlags :

```
struct FShowFlags
{
    bool bParticles = true;
};
```

Toolbar의 Show Flags 팝업에는 `Particles` 체크박스가 있고, `RenderCollector` 는 `bParticles == false` 일 때 Particle proxy 수집을 건너뛴다. 이 방식은 Component Tick 자체를 막지 않고 렌더 수집 단계에서만 제외하므로, 에디터에서 시뮬레이션 상태와 렌더 표시 여부를 분리할 수 있다.

`UParticleSystemComponent` 에도 `bShowParticles` 가 있어 개별 Component 단위 표시를 제어한다. 전체 Viewport 표시 여부는 Show Flag가 담당하고, 개별 Actor/Component 표시 여부는 Component visibility가 담당한다.

8. Stat

Particle Stat은 `FParticleStats` 가 담당한다.

파일:

- `KraftonEngine/Source/Engine/Profiling/ParticleStats.h`
- `KraftonEngine/Source/Engine/Profiling/ParticleStats.cpp`
- `KraftonEngine/Source/Editor/Subsystem/OverlayStatSystem.cpp`
- `KraftonEngine/Source/Editor/UI/EditorConsoleWidget.cpp`

수집 항목:

- Component 수
- Emitter 수

- Active Particle 수
- Max Particle 수
- Simulation memory
- Packed render memory
- 타입별 통계

- Sprite

- Mesh

- Beam

- Ribbon

- CPU 시뮬레이션 구간별 시간
- 렌더 패킹/업로드 구간별 시간

주요 타이머:

- ComponentTick
- InitParticles
- BuildInstances
- EmitterTick
- Spawn
- ParticleUpdate
- BuildRenderData
- UpdateDynamicData
- UpdateMesh
- SortEmitters
- PackSprites
- PackMeshes
- PackBeams
- PackRibbons
- UploadSpriteBuffers
- UploadMeshInstances
- UploadBeamBuffers
- UploadRibbonBuffers

콘솔 명령:

```
stat particle
```

이 명령은 Overlay Stat System의 Particle 표시를 toggle한다. Overlay에는 `Stat Particle` 창으로 파티클 수, 메모리, 타입별 Pack 수, CPU/Render 타이밍이 표시된다.

9. LOD

Particle LOD는 `UParticleSystem`의 거리 배열과 각 Emitter의 `LODLevels`로 구성된다.

주요 흐름:

1. `UParticleSystem::NormalizeLODData()`가 시스템 LOD 수와 Emitter LOD 수를 맞춘다.
2. `UParticleSystemComponent::InitParticles()`가 `LODDistances`를 캐싱한다.
3. Tick에서 `DecideLODLevel()`이 카메라와 Component 위치의 거리를 계산한다.
4. 각 `FParticleEmitterInstance`에 선택된 LOD index를 전달한다.
5. Emitter Instance는 해당 LOD의 Module 목록으로 Spawn/Update를 수행한다.

`ForcedLODLevel`을 사용하면 에디터나 테스트에서 특정 LOD를 강제로 선택할 수 있다.

거리 기반 LOD는 파티클 수, Spawn Rate, Material, TypeData를 LOD별로 다르게 만들 수 있으므로, 원거리 이펙트의 CPU/GPU 비용을 줄이는 데 유효하다. 단, LOD 전환 시 파티클 상태가 갑자기 바뀔 수 있으므로 실제 콘텐츠에서는 거리 임계값과 모듈 값 차이를 신중히 조정해야 한다.

10. Collision & Event

10.1 Collision

Collision은 `UParticleModuleCollision`이 담당한다.

주요 설정:

- Trace Channel
- Response Mode

- Bounce

- Stop

- Kill

- Damping Factor
- Collision Offset
- Collision Radius Scale
- Max Collisions

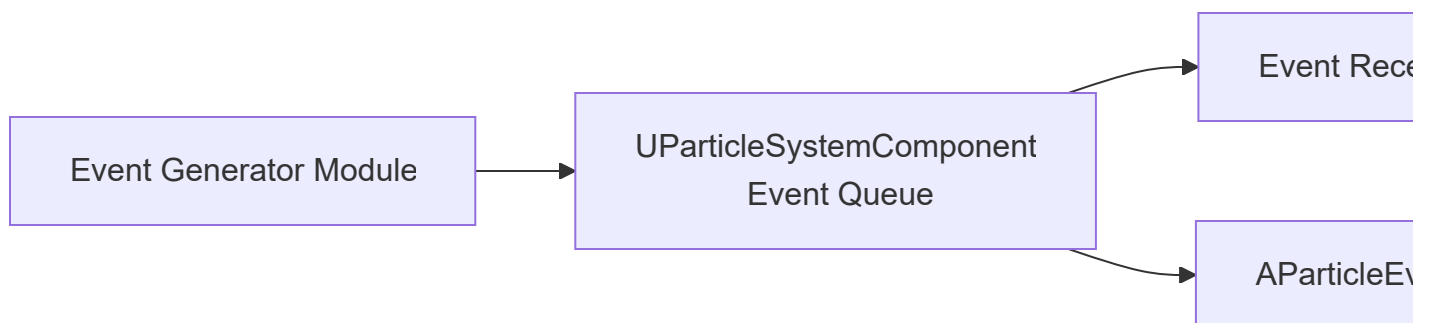
Collision Module은 FinalUpdate 단계에서 파티클 이동 구간을 trace하고 충돌 결과에 따라 속도 반사, 정지, 제거를 수행한다. 충돌이 발생하면 `FParticleEventCollideData` 가 `UParticleSystemComponent::CollisionEvents` 에 쌓인다.

10.2 Event

Event 흐름은 다음 구조를 사용한다.

- `UParticleModuleEventGenerator`
- `UParticleModuleEventReceiverBase`
- `UParticleModuleEventReceiverSpawn`
- `AParticleEventManager`
- `UParticleSystemComponent::SpawnEvents`
- `UParticleSystemComponent::DeathEvents`
- `UParticleSystemComponent::CollisionEvents`
- `UParticleSystemComponent::BurstEvents`

흐름:



Emitter 내부에서 생성된 이벤트는 Component 큐에 모이고, 같은 Component 안의 Event Receiver Module이 먼저 처리할 수 있다. 이후 월드의 `AParticleEventManager` 로 전달되어 외부 시스템과 연결된다.

11. Symbol Server

Symbol Server는 빌드 산출물의 PDB를 공유 symbol store에 등록하고, 필요하면 Source Server 정보를 PDB에 삽입하는 구조다.

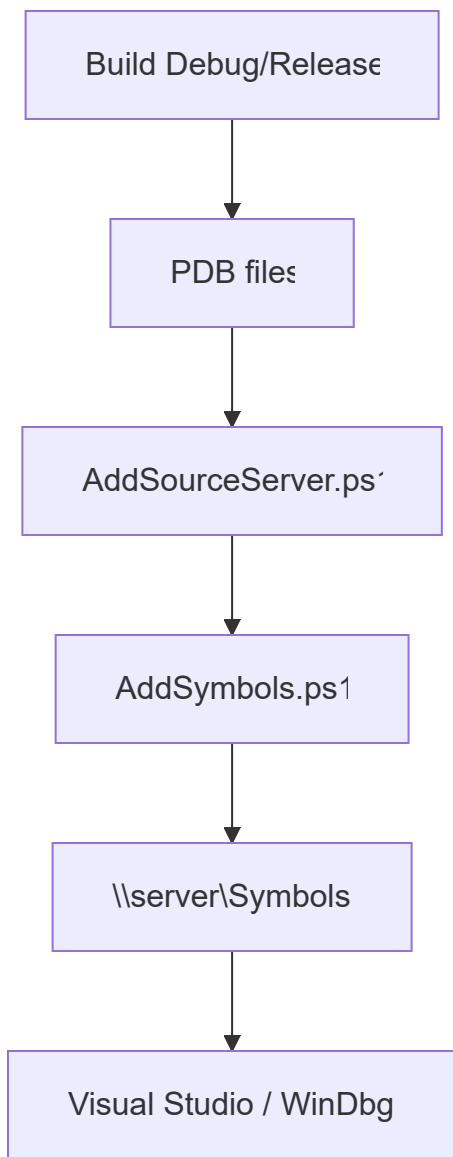
관련 스크립트:

- `Scripts/AddSymbols.ps1`
- `Scripts/AddSourceServer.ps1`

사용 도구:

- `symstore.exe` : PDB를 symbol store에 등록
- `src tool.exe` : PDB에 기록된 source file 목록 확인
- `pdbstr.exe` : PDB의 `srcsrv` stream에 Source Server 정보 기록

기본 흐름:



기본 등록 예시:

```
.\Scripts\AddSymbols.ps1 `

-Configuration Debug `

-StorePath "\\192.168.xxx.yyy\symbols" `

-ProductName "NipsEngine" `

-BuildName "Week12-Debug"
```

Source Server까지 포함하는 예시:


```
.\Scripts\AddSymbols.ps1 `

-Configuration Debug `

-StorePath "\\192.168.xxx.yyy\symbols" `

-EnableSourceServer `

-SourceRepo "\\192.168.xxx.yyy\SourceRepos\Week12.git" `

-SourceMode Snapshot `

-SourceSnapshotLocalRoot "C:\SourceRepos\Snapshots" `

-SourceSnapshotShareRoot "\\192.168.xxx.yyy\SourceRepos\Snapshots"
```

운영 시 주의할 점:

- Windows SDK Debugging Tools가 설치되어 있어야 한다.
- 빌드 설정에서 PDB 생성이 켜져 있어야 한다.
- symbol share는 모든 팀원이 읽을 수 있어야 하고, 빌드 등록자는 쓸 수 있어야 한다.
- Source Server를 사용할 경우 PDB에 기록된 commit과 source snapshot/share가 일치해야 한다.
- **Snapshot** 모드는 디버거가 Git 명령을 실행하지 않아도 되므로 팀 환경에서 더 예측 가능하다.

12. 확장성과 유지보수 관점

이 구조에서 가장 중요한 설계 원칙은 에셋, 시뮬레이션, 렌더링의 책임 분리다.

좋은 확장 방향:

- 새 파티클 동작은 **UParticleModule** 파생 클래스로 추가한다.
- 새 Emitter 렌더 타입은 **UParticleModuleTypeDataBase**, 전용 Emitter Instance, 전용 ReplayData, 전용 Packer를 함께 추가한다.
- 렌더러는 Component 내부 상태를 직접 참조하지 않고 ReplayData만 소비한다.
- Stat은 시뮬레이션, 렌더 데이터 생성, GPU 업로드를 분리해서 기록한다.
- LOD별 Module 목록은 런타임 Tick 중 동적 검색하지 않고 사전 분류한다.

피해야 할 방향:

- Module이 Component나 Scene Proxy의 내부 구현을 직접 변경하는 구조
- 렌더러가 `FParticleEmitterInstance::ParticleData` 를 직접 참조하는 구조
- Emitter Type마다 Component Tick에 큰 `if/else` 를 계속 추가하는 구조
- Collision/Event를 전역 static queue에 숨겨서 처리하는 구조
- LOD 전환을 렌더러에서 임의로 결정하는 구조

현재 설계는 CPU 기반 Cascade 스타일 구현에는 적합하다. 장기적으로 GPU particle simulation, Niagara식 graph 기반 authoring, 멀티스레드 simulation을 추가하려면

`FParticleEmitterInstance` 와 `FDynamicEmitterReplayDataBase` 사이에 job scheduling 및 double-buffered snapshot 계층을 추가하는 편이 바람직하다. 지금 구조를 유지하면 그 확장도 Component와 Scene Proxy의 경계를 크게 깨지 않고 진행할 수 있다.

13. 주요 파일 요약

| 영역 | 파일 |

|---|---|

| Particle 에셋 | `KraftonEngine/Source/Engine/Particle/ParticleSystem.*` |

| Emitter 템플릿 | `KraftonEngine/Source/Engine/Particle/ParticleEmitter.*` |

| LOD | `KraftonEngine/Source/Engine/Particle/ParticleLODLevel.*` |

| Module | `KraftonEngine/Source/Engine/Particle/ParticleModule.h`,
`ParticleModules.cpp` |

| Emitter Instance | `KraftonEngine/Source/Engine/Particle/ParticleEmitterInstances.*` |

| Beam Instance | `KraftonEngine/Source/Engine/Particle/ParticleBeamInstances.*` |

| Ribbon Instance | `KraftonEngine/Source/Engine/Particle/RibbonEmitterInstance.*` |

| Component | `KraftonEngine/Source/Engine/Component/ParticleSystemComponent.*` |

| Render Dynamic Data |

`KraftonEngine/Source/Engine/Render/Particle/ParticleDynamicData.*` |

| Render Proxy | `KraftonEngine/Source/Engine/Render/Proxy/ParticleSystemSceneProxy.*`

|

| Vertex Types | `KraftonEngine/Source/Engine/Render/Types/VertexTypes.h` |

| Show Flag | `KraftonEngine/Source/Engine/Render/Types/ViewTypes.h`,
`ViewportToolbar.cpp`, `RenderCollector.cpp` |

| Stat | `KraftonEngine/Source/Engine/Profiling/ParticleStats.*`,
`OverlayStatSystem.cpp` |

| Particle Editor | `KraftonEngine/Source/Editor/UI/Asset/ParticleEditorWidget.*` |

| Symbol Server | `Scripts/AddSymbols.ps1`, `Scripts/AddSourceServer.ps1` |

14. 검증 체크리스트

- Content Browser에서 Particle System 에셋 생성 가능
- Particle System Editor에서 Emitter 추가/삭제/이름 변경 가능
- Sprite Emitter가 Preview와 Level Viewport에서 렌더링됨
- Mesh Emitter가 Static Mesh instance로 렌더링됨
- Material/SubUV/Screen Alignment/Sort Mode 변경이 렌더 결과에 반영됨
- Show Flags의 `Particles` 체크 해제 시 파티클 렌더링이 제외됨
- `stat particle` 로 Overlay Stat 표시/숨김 가능
- LOD 거리 변경 시 Component의 선택 LOD가 변경됨
- Collision Module에서 Bounce/Stop/Kill 반응 확인
- Collision Event가 Component/EventManager로 전달됨
- Beam/Ribbon TypeData 선택 시 전용 Emitter Instance와 RenderData 경로 사용
- `Scripts/AddSymbols.ps1` 로 PDB가 symbol store에 등록됨
- Source Server 사용 시 PDB에 `srcsrv` stream이 기록됨